

Trabajo Practico Integrador

Paradigmas y Lenguajes



Grupo: 48

Integrantes:

Ortiz, Santiago Tomas

Fase 1

Requerimiento 1: Transición

```
(defun transicion (color-actual cambiar-a)
  (cond ((and (equalp color-actual 'en-rojo)(equalp cambiar-a 'verde))(list color-actual "cambiar-a-verde"))
        ((and (equalp color-actual 'en-verde)(equalp cambiar-a 'amarillo))(list color-actual "cambiar-a-amarillo"))
        ((and (equalp color-actual 'en-amarillo) (equalp cambiar-a 'rojo))(list color-actual "cambiar-a-rojo"))
        (t(list color-actual 'accion-por-defecto))))
```

El objetivo de esta función es que mediante el color actual y el color al que se va a cambiar, evalúe si es posible el cambio, devolviendo una lista con ambos colores.

Al principio la “pensamos” en como “filtrar” los casos no válidos, pero al ver que son demasiados y que era mas sencillo solo poner los posibles, cambiamos el diseño al de la imagen.

Creemos que es un buen diseño ya que cualquier caso no valido lo atrapa con “acción-por-defecto”, y en caso de ser valido devuelve la lista con ambos colores.

Requerimiento 2: timer

```
(defun timer(tiempo_unix)
  (cond
    ((< (mod tiempo_unix 216) 90) 'rojo)
    ((< (mod tiempo_unix 216) 210) 'verde)
    (t 'amarillo)))
```

El objetivo de esta función es que con el tiempo que se le pasa por parámetro, mediante el uso del mod o resto de dividir ese numero o tiempo por 216 (un ciclo completo), vaya “descomponiéndose” el numero y entrando en las diferentes condiciones, si es menor a 90 es rojo, si es menor a 210 es verde, y si no es ninguna, amarillo.

Tuvimos que cambiar las condiciones por comprender mal el ciclo en un principio.

Creemos que es un buen diseño ya que, al hacer el mod, el resto no puede ser menor que 0 y no puede ser mayor o igual que 216, entonces siempre entraría en algunas de las condiciones.

Requerimiento 3: Auditoria

```
(defun auditoria (color-anterior color-nuevo)
  (format t "Tiempo ~A: La luz ha cambiado de ~A a ~AX"
    (- (get-universal-time) 2208988800) (car(transicion color-anterior color-nuevo)) (cadr(transicion color-anterior color-nuevo)))))
```

El objetivo de esta función es mostrar por pantalla un registro donde se muestra el tiempo (en tiempo Unix) en que se hizo el cambio de un color a otro.

Tuvimos un problema con la validación de colores en un principio, por ello, decidimos usar la función anterior transición para validar si el cambio de un color a otro era válido, usando car, cdr y sus combinaciones, para sacar los elementos de la lista creada por transición.

Creemos que es buen diseño, ya que por como se explico lo de la validación, no se va a hacer un cambio invalido, también decidimos usar el (- (get-universal-time) 2208988800) ya que en un principio no comprendíamos o no le encontrábamos sentido pasar el tiempo por parámetro, entonces buscando una alternativa encontramos esa “operación”, que hace la diferencia de segundos entre el origen de tiempo de Common Lisp (1900) y la época Unix (1970), sería como “sincronizar” ambos tiempos, para así luego poder mostrarlo por pantalla.

Requerimiento 4:

```
(defun duracion-ciclo()
  (+ 90 6 120))
```

El objetivo de esta función es calcular el ciclo completo, el 90 es el rojo, el 6 el amarillo y el 120 el verde.

```
(defun recomendacion-ciclo()
  (if (and (> (calcularCiclo) 35)
        (< (calcularCiclo) 150))
      (print "Esta en el rango optimo")
      "No esta en el rango optimo"))
```

El objetivo de esta función es con el ciclo completo calculado anteriormente se determine si esta en un rango optimo entre 35 y 150.

Siempre va a estar fuera de rango el ciclo.

Requerimiento 5:

```
(defun ciclos-por-tiempo(duracionMinutos)
  (format t "La cantidad de ciclos es: ~A%" (truncate(/ (* 60 duracionMinutos) 216))))
```

El objetivo de esta función es calcular la cantidad de ciclos completos que entran en el tiempo que se le ingresa.

Creemos que es buen diseño ya que se calcula todo dentro de la función sin necesidad de crear funciones externas, y con el truncate nos aseguramos de que el número sea entero.

Requerimiento 6:

```
(defun porcentajeColor(segundos)
  (float(* (/ segundos 3600) 100)))
```

El objetivo de esta función es calcular porcentajes

```
(defun intervalosCiclo(inicio)
  (cond
    ((and(< inicio 90)(<(- (mod 3600 216) (- 90 inicio))120)) (list (- 90 inicio) (- (mod 3600 216) (- 90 inicio)) 0))
    ((< inicio 90)(list (+ (- 90 inicio)(- (mod 3600 216) (- 90 inicio) 120 6)) 120 6))
    ((< inicio 210)(list (- (mod 3600 216) (+ (- 210 inicio) 6)) (- 210 inicio) 6))
    ((and(>= inicio 210) (> (- (mod 3600 216) (- 216 inicio)) 90))(list 90 (- (mod 3600 216) (- 216 inicio) 90) (- 216 inicio)))
    (t (list(- (mod 3600 216) (- 216 inicio)) 0 (- 216 inicio)))))
```

El objetivo de esta función es ver donde “inicia” y termina el tiempo utilizando los 144 segundos restantes de los 16,6 ciclos. Se le pasa por parámetro el mod del tiempo ingresado por 216, y mediante eso hay distintas condiciones para varios casos:

El primer caso es para cuando no inicia en el rojo y el verde no llega a terminar, entonces el programa va a hacer rojo – inicio, y (mod 3600 216) – (rojo – inicio), lo que nos daría una lista donde (rojo, verde sin terminar, 0) ya que el amarillo nunca se llegó a consumir.

El segundo caso es también para cuando inicia en rojo, pero el (mod 3600 216) o 144 segundos son suficientes para completar el ciclo, entonces el verde y el amarillo se llegan a completar y avanzan un poco más llegando a rojo, entonces la operación que se realiza ahí es para asignar esos segundos que se “paso” y dárselos al rojo.

El tercer caso es cuando inicia en el verde. El programa calcula cuanto tiempo le queda a ese verde. Y como el tiempo restante de la hora no es suficiente para completar el ciclo, termina en el rojo del ciclo siguiente. El amarillo queda fijo en 6, se calcula el tiempo del verde y el rojo recibe el excedente.

El cuarto caso es cuando inicia en amarillo y no llega a completarse el verde, el rojo del ciclo siguiente se consume completo, pero el verde queda a la mitad, entonces la operación es para calcular cuánto verde se consumió.

El quinto caso es cuando inicia en amarillo, pero no llega al verde, solo recorre un poco el rojo, por eso el verde es 0.

```
(defun mostrarPorcentajes(hora-unix)
  (list
    'rojo (porcentajeColor (+ (* 16 90) (car (intervalosCiclo (mod hora-unix 216))))))
    'verde (porcentajeColor (+ (* 16 120) (cadr (intervalosCiclo (mod hora-unix 216))))))
    'amarillo (porcentajeColor (+ (* 16 6) (caddr (intervalosCiclo (mod hora-unix 216))))))
```

El objetivo de esta función es mostrar los porcentajes de cada color según el tiempo ingresado.

Se hace una lista con los porcentajes de cada color, se multiplica 16 a los segundos de cada color ya que son los ciclos completos en 1 hora, y los 0,6 restantes se evalúan en intervalos ciclos, para ver donde termina.

Como intervalosCiclos te devuelve una lista con 3 elementos, de la forma (rojo, verde, amarillo), para utilizarlos se debe usar car, cdr y sus combinaciones.

Finalmente se suman y se hace el porcentaje con la función que creamos antes.

Iteración 1:

Extensión 1:

```
(defun transicionIntermitencia (color-actual cambiar-a)
  (cond ((and (equalp color-actual 'en-rojo-intermitente)(equalp cambiar-a 'verde))(list color-actual "cambiar-a-verde"))
        ((and (equalp color-actual 'en-verde-intermitente)(equalp cambiar-a 'amarillo))(list color-actual "cambiar-a-amarillo"))
        ((and (equalp color-actual 'en-amarillo-intermitente)(equalp cambiar-a 'rojo))(list color-actual "cambiar-a-rojo"))
        ((and (equalp color-actual 'en-rojo)(equalp cambiar-a 'rojo-intermitente))(list color-actual "cambiar-a-rojo-intermitente"))
        ((and (equalp color-actual 'en-verde)(equalp cambiar-a 'verde-intermitente))(list color-actual "cambiar-a-verde-intermitente"))
        ((and (equalp color-actual 'en-amarillo)(equalp cambiar-a 'amarillo-intermitente))(list color-actual "cambiar-a-amarillo-intermitente"))
        (t (list color-actual 'accion-por-defecto))))
```

Acá la lógica de transición del requerimiento 1 es la misma, solo añadimos los casos donde un color pase a su intermitente.

Como, por ejemplo, (amarillo, "cambiar-a-amarillo-intermitente")

```
(defun timer-intermitencia(tiempo_unix)
  (cond
    ((<= (mod tiempo_unix 225) 89) 'rojo)
    ((<= (mod tiempo_unix 225) 92) 'rojo-intermitente)
    ((<= (mod tiempo_unix 225) 212) 'verde)
    ((<= (mod tiempo_unix 225) 215) 'verde-intermitente)
    ((<= (mod tiempo_unix 225) 221) 'amarillo)
    (t 'amarillo-intermitente)))
```

Lo mismo de antes, la lógica del timer es la misma solo que se añaden más casos.

En esta función, se le añaden 3 segundos mas de intermitencia a cada color, por ejemplo, 90 segundos del rojo, y 3 de intermitencia, $90 + 3 = 93$. Y así con los otros colores.

Creemos que es un buen diseño ya que, al hacer el mod, el resto no puede ser menor que 0 y no puede ser mayor o igual que 225, entonces siempre entraría en algunas de las condiciones.

```
(defun duracion-ciclo-intermitencia()
  (+ 90 6 120 9))      ;; 9 segundos de intermitencia
```

El objetivo de esta función es calcular el ciclo completo junto con la intermitencia de cada color.

```
(defun intervalos-ciclo-intermitencia (inicio)
  (cond
    ((<= inicio 89) (list (- 90 inicio) 3 120 3 6 3))
    ((<= inicio 92) (list 0 (- 3 (- inicio 90)) 120 3 6 3))
    ((<= inicio 212) (list 0 0 (- 120 (- inicio 93)) 3 6 3))
    ((<= inicio 215) (list 0 0 0 (- 3 (- inicio 213)) 6 3))
    ((<= inicio 221) (list 0 0 0 0 (- 6 (- inicio 216)) 3))
    (t (list 0 0 0 0 0 (- 3 (- inicio 222))))))
```

Con la intermitencia añadida, esta función se vuelve más simple.

Calcula cuantos segundos de cada color habrá dentro del ultimo ciclo, se le llega el mod de la hora unix por 225, y con eso determina el inicio del ciclo.

Cuando un color ya se consumió por completo, se le coloca 0 dependiendo de donde arranque, por ejemplo, si inicia en rojo intermitente, el rojo ya se utilizo y solo calcula los restantes.

Esto no significa que el rojo no aparezca en la hora completa, sino que solo que cuando arranca el último ciclo ya no queda rojo por usar.

```
(defun mostrar-porcentajes-intermitencia (hora-unix)
  (list
    'rojo (porcentaje-color (+ (* 15 90) (car (intervalos-ciclo-intermitencia (mod hora-unix 225))) (- 90 (car (intervalos-ciclo-intermitencia (mod hora-unix 225))))))
    'rojo-intermitente (porcentaje-color (+ (* 15 3) (cadr (intervalos-ciclo-intermitencia (mod hora-unix 225))) (- 3 (cadr (intervalos-ciclo-intermitencia (mod hora-unix 225))))))
    'verde (porcentaje-color (+ (* 15 120) (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))) (- 120 (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))))))
    'verde-intermitente (porcentaje-color (+ (* 15 3) (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))) (- 3 (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))))))
    'amarillo (porcentaje-color (+ (* 15 6) (car (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))) (- 6 (car (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))))))
    'amarillo-intermitente (porcentaje-color (+ (* 15 3) (cadr (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225))) (- 3 (cadr (caddr (intervalos-ciclo-intermitencia (mod hora-unix 225)))))))))
```

En esta función se calculan los porcentajes, se multiplica 15 por la duración total de los colores, ya que al hacer (/ 3600 225) nos da 16 justo, son 16 ciclos completos, así que si nosotros calculamos el último ciclo con intervalos-ciclo-intermitencia, se utilizan los 15 ciclos completos anteriores y se le suma el ultimo.

Luego, se hace la diferencia entre el total de la duración del color por los segundos del mismo, para ver en donde termina.

Pero el ciclo al ser entero, siempre va a terminar donde inicia. Por esta razón, los segundos que faltan al inicio de un color se recuperan al final, obteniendo siempre los mismos porcentajes sin importar la hora inicial.

Extension 2:

```
(defun informe (color-actual cambiar-a)
  (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output :if-exists :append)
    (format stream "Informe de Ejecución del Sistema Semafórico~%")
    (format stream "=====~%")
    (format stream (auditoriaInforme color-actual cambiar-a))
    (format stream "~% --- Fin del Informe --- ~%")))
```

El objetivo de esta función es que se registre cada cambio en un archivo de texto

“Nosotros” utilizamos la biblioteca local time que más adelante explicaremos.

Y nos percatamos que el formato que solicitaba era similar a la auditoria que hicimos antes, solo la ajustamos un poco.

Nosotros queríamos que se escriba abajo cada cambio, y por eso, utilizamos el if-exist :append para que no escriba por encima y se escriba directamente a continuación del último cambio.

```
(defun auditoriaInforme (color-anterior color-nuevo)
  (format nil "Tiempo ~A: La luz ha cambiado de ~A a ~A-%" (local-time:format-timestring nil (local-time:now) :format '(:year "-" :month "-" :day " " :hour ":" :min ":" :sec))
    (car (transicion color-anterior color-nuevo)) (cadr(transicion color-anterior color-nuevo))))
```

En esta función tuvimos que cambiar el format t por format nil, ya que al ser format t devolvía NIL al final, cortando la ejecución haciendo que no se escriba nada en el archivo.

Fase 2

```
(defun auditoria (color-anterior color-nuevo)
  (format t "Tiempo ~A: La luz ha cambiado de ~A a ~A-%" (local-time:format-timestring nil (local-time:now) :format '(:year "-" :month "-" :day " " :hour ":" :min ":" :sec))
    (car (transicion color-anterior color-nuevo)) (cadr(transicion color-anterior color-nuevo))))
```

Decidimos utilizar local-time ya que fue lo que pudimos descargar y utilizar de manera sencilla

Local-time funciona dándonos un tiempo local, junto con el format nos devuelve mostrándonos el año mes día hora minuto y segundo.

Bitácora de depuración

Commit 5847267

```
@@ -32,8 +32,8 @@
32 32      (defun timer(tiempo_unix)
33 33          (cond
34 34              ((< (mod tiempo_unix 216) 90) 'rojo)
35 -              ((< (mod tiempo_unix 216) 96) 'amarillo)
36 -              (t 'verde)))
35 +              ((< (mod tiempo_unix 216) 210) 'verde)
36 +              (t 'amarillo)))
37 37
38 38      #|;Requerimiento 3: Temporizador Automático;
39 39      ;; =====
```

Hubo un error en como funcionaba el ciclo, pensamos que era de rojo->amarillo ->verde.

Commit 498abd7

```
8 8      ;; =====
9 9
10 10      (defun transicion (color-actual cambiar-a)
11 -          (cond ((not(or(and (equalp color-actual 'en-rojo)(equalp cambiar-a 'amarillo))
12 -                          (and (equalp color-actual 'en-amarillo)(equalp cambiar-a 'verde))
13 -                          (and (equalp color-actual 'en-verde) (equalp cambiar-a 'rojo))))))
11 +          (cond ((not(or(and (equalp color-actual 'en-rojo)(equalp cambiar-a 'verde))
12 +                          (and (equalp color-actual 'en-verde)(equalp cambiar-a 'amarillo))
13 +                          (and (equalp color-actual 'en-amarillo) (equalp cambiar-a 'rojo))))))
14 14          (list color-actual 'accion-por-defecto))
15 15
16 16          ((equalp cambiar-a 'rojo)(list color-actual "cambiar-a-rojo"))
```

Acá se juntan dos errores, primero el del ciclo anteriormente mencionado, y otro mas con el uso de not (or (and, no estaba mal implementado, pero se podía optimizar y hacer el código mas corto, evaluando solo los casos validos en vez de usar un not y or para lo mismo.

```
(defun transicion (color-actual cambiar-a)
  (cond ((not(or(and (equalp color-actual 'en-rojo)(equalp cambiar-a 'verde))
    (and (equalp color-actual 'en-verde)(equalp cambiar-a 'amarillo))
    (and (equalp color-actual 'en-amarillo) (equalp cambiar-a 'rojo))))))
    (list color-actual 'accion-por-defecto))

    ((equalp cambiar-a 'rojo)(list color-actual "cambiar-a-rojo"))
    ((equalp cambiar-a 'amarillo)(list color-actual "cambiar-a-amarillo"))
    ((equalp cambiar-a 'verde)(list color-actual "cambiar-a-verde"))
    (t (list color-actual 'accion-por-defecto))))

  (cond ((and (equalp color-actual 'en-rojo)(equalp cambiar-a 'verde))(list color-actual "cambiar-a-verde"))
    ((and (equalp color-actual 'en-verde)(equalp cambiar-a 'amarillo))(list color-actual "cambiar-a-amarillo"))
    ((and (equalp color-actual 'en-amarillo) (equalp cambiar-a 'rojo))(list color-actual "cambiar-a-rojo"))
    (t(list color-actual 'accion-por-defecto))))
```


Errores con el requerimiento 6:

```
Break 2 [4]> (defun intervalosCiclo(inicio)
  (cond
    ((<= inicio 90)(list (- 90 inicio) 120 (- (mod 3600 216) (- 90 inicio) 120)))
    ((<= inicio 210) (list (- (mod 3600 216) (- 210 inicio) 6) (- 210 inicio) 6))
    (t (list(- (mod 3600 216) (- 216 inicio)) 0 (- 216 inicio)))))

INTERVALOSCICLO
Break 2 [4]> (intervalosCiclo 90)

(0 120 24)
Break 2 [4]> (intervalosCiclo 209)

(137 1 6)
```

Al hacer el 209 nos dimos cuenta que el rojo daba más de 90

Eso fue ya que al hacer 209, se consume todo el rojo y terminaría en el verde un poco adelante, el problema de esta función es que no tuvimos en cuenta los excedentes, entonces aquí el rojo tiene la parte que le corresponde al verde

La solución fue poner un and para verificar si excede

```
(defun intervalosCiclo(inicio)
  (cond
    ((and(< inicio 90)(<(- (mod 3600 216) (- 90 inicio))120)) (list (- 90 inicio) (- (mod 3600 216) (- 90 inicio)) 0))
    ((< inicio 90)(list (+ (- 90 inicio)(- (mod 3600 216) (- 90 inicio) 120 6)) 120 6))
    ((< inicio 210)(list 90 (+ (- 210 inicio) (- (mod 3600 216) (- 210 inicio) 6 90)) 6))
    ((and(>= inicio 210) (> (- (mod 3600 216) (- 216 inicio)) 90))(list 90 (- (mod 3600 216) (- 216 inicio) 90) (- 216 inicio)))
    (t (list(- (mod 3600 216) (- 216 inicio)) 0 (- 216 inicio)))))
```

Lo utilizamos con el excedente del verde y con el excedente del rojo

```
Break 1 [3]> (defun intervalosCiclo(inicio)
  (cond
    ((and(< inicio 90)(>(- (mod 3600 216) (- 90 inicio) 6)120)) (list (+ (- 90 inicio) (- (mod 3600 216) (- 90 inicio)120 6))120 6))
    ((< inicio 90)(list (- 90 inicio) 120 (- (mod 3600 216) (- 90 inicio) 120)))
    ((< inicio 210) (list (- (mod 3600 216) (- 210 inicio) 6) (- 210 inicio) 6))
    ((and(>= inicio 210) (> (- (mod 3600 216) (- 216 inicio)) 90))(list 90 (- (mod 3600 216) (- 216 inicio) 90) (- 216 inicio)))
    (t (list(- (mod 3600 216) (- 216 inicio)) 0 (- 216 inicio)))))

INTERVALOSCICLO
Break 1 [3]> (intervalosCiclo 5)

(85 120 -61)
```

Daba negativo el amarillo

El error estaba en el caso de en el primer and, justo en la condición de > 120

Entonces al hacer el caso de prueba con 5, se hace la comparación $5 > 120$, lo que da falso y sigue a la siguiente condición.

Entonces entra en la de ($< \text{inicio } 90$), ahí el amarillo se calcula mal y da -61

El otro error también es que, al entrar en esa condición, que es donde inicia en rojo y completa todo el ciclo, ese 120 estaría mal.

El error estaba justamente en lo que se ejecuta a continuación de esa condición, por lo que, la solución fue hacer que no se sume nada, sino que se reste bien el 90 al inicio, y la resta de 144 al rojo, y como nunca llega al amarillo, que sea 0.

```
(defun intervalosCiclo(inicio)
  (cond
    ((and(< inicio 90)(<(- (mod 3600 216) (- 90 inicio))120)) (list (- 90 inicio) (- (mod 3600 216) (- 90 inicio)) 0))
    ((< inicio 90)(list (+ (- 90 inicio)(- (mod 3600 216) (- 90 inicio) 120 6)) 120 6))
    ((< inicio 210)(list 90 (+ (- 210 inicio) (- (mod 3600 216) (- 210 inicio) 6 90)) 6))
    ((and(>= inicio 210) (> (- (mod 3600 216) (- 216 inicio)) 90))(list 90 (- (mod 3600 216) (- 216 inicio) 90) (- 216 inicio)))
    (t (list(- (mod 3600 216) (- 216 inicio)) 0 (- 216 inicio)))))
```

```
Break 2 [4]> (intervalosCiclo1 0)
(intervalosCiclo1 52)
(intervalosCiclo1 85)
(intervalosCiclo1 90)
(intervalosCiclo1 100)
(intervalosCiclo1 150)
(intervalosCiclo1 210)
(intervalosCiclo1 212)
(intervalosCiclo1 215)
```

```
(90 54 0)
Break 2 [4]>
(38 106 0)
Break 2 [4]>
(18 120 6)
Break 2 [4]>
(90 48 6)
Break 2 [4]>
(90 48 6)
Break 2 [4]>
(90 48 6)
Break 2 [4]>
(90 48 6)
Break 2 [4]>
(90 50 4)
Break 2 [4]>
(90 53 1)
```

Haciendo casos de prueba de la función del requerimiento 6, nos dimos cuenta que la función tendía a devolver de forma constante el valor rojo 90 y 6 del amarillo, cosa que nos pareció rara.

El error estaba en que en la condición donde inicia en verde, nosotros asumimos que siempre iba a terminar en el verde del ciclo siguiente, entonces pusimos los valores fijos del rojo y el amarillo.

Entonces al no completarlo, quedan esos valores “falsos” de 90 y 6.

La forma de solucionarlo fue que en vez de que se usen valores fijos, se calculen con el (mod 3600 216) anteriormente mencionado y restando 210 y 6 a inicio.

```
(defun intervalosCiclo(inicio)
  (cond
    ((and(< inicio 90)(<(- (mod 3600 216) (- 90 inicio))120)) (list (- 90 inicio) (- (mod 3600 216) (- 90 inicio)) 0))
    ((< inicio 90)(list (+ (- 90 inicio)(- (mod 3600 216) (- 90 inicio) 120 6)) 120 6))
    ((< inicio 210)(list (- (mod 3600 216) (+ (- 210 inicio) 6)) (- 210 inicio) 6))
    ((and(>= inicio 210) (> (- (mod 3600 216) (- 216 inicio)) 90))(list 90 (- (mod 3600 216) (- 216 inicio) 90) (- 216 inicio)))
    (t (list(- (mod 3600 216) (- 216 inicio)) 0 (- 216 inicio)))))
```

Ocaml

OCaml (Objective Categorical Abstract Machine Language) es un lenguaje de programación de nivel industrial que combina tres paradigmas:

- Funcional
- Imperativo
- Orientado a objetos

Es conocido por su alta seguridad, velocidad de ejecución y gestión de memoria.

OCaml combina la

Los sectores donde se utiliza OCaml son:

- Finanzas
- Desarrollo web
- Análisis de datos

Las empresas que utilizan OCaml son varias, entre ellas:

- Facebook
- Bloomberg
- CEA-List
- Cryptosense

Las empresas que utilizan OCaml es mayormente por su alta seguridad y su buen rendimiento, muchas compañías utilizan este lenguaje para guardar sus datos y operar de forma segura y eficiente.

Si les tocó OCAML:

- 1. Al igual que Haskell, OCaml usa tipos estáticos, pero cuenta con *Inferencia de Tipos*. ¿Tuvieron que declarar explícitamente de qué tipo eran las funciones o el compilador lo dedujo solo? Muestren cómo lo interpreta el entorno.
- 2. OCaml es un lenguaje "orientado a expresiones". ¿Cómo cambia esto la estructura de control de flujos (como el uso de `match...with`) comparado con los condicionales de Lisp?

1) Al hacer las funciones, no tuvimos que declarar explícitamente de que tipo eran las funciones, pero si tuvimos que definir el tipo de dato, ya que, sin eso, el compilador no reconoce esos valores.

<pre>1 let transicion coloractual cambiara = 2 match (coloractual, cambiara) with 3 (EnRojo, Verde) -> (coloractual,"cambiar a verde") 4 (EnVerde, Amarillo) -> (coloractual, "cambiar a amarillo") 5 (EnAmarillo, Rojo) -> (coloractual, "cambiar a rojo") 6 _ -> (coloractual, "Accion por defecto") 7 8 9 let prueba1 = transicion EnVerde Amarillo 10 let prueba2 = transicion EnRojo Verde 11 let prueba3 = transicion Verde Verde 12 let prueba4 = transicion EnAmarillo Rojo</pre>	Output Line 3, characters 3-9: Error: Unbound constructor EnRojo
---	--

Eso pasa si no definimos el tipo de dato antes, el compilador no reconoce el “constructor” o no sabe que es “EnRojo” porque no fue definido.

Definiendo el tipo de dato “color”, puede compilar

<pre>type color = EnRojo Verde Rojo EnVerde Amarillo EnAmarillo let transicion coloractual cambiara = match (coloractual, cambiara) with (EnRojo, Verde) -> (coloractual,"cambiar a verde") (EnVerde, Amarillo) -> (coloractual, "cambiar a amarillo") (EnAmarillo, Rojo) -> (coloractual, "cambiar a rojo") _ -> (coloractual, "Accion por defecto") let prueba1 = transicion EnVerde Amarillo let prueba2 = transicion EnRojo Verde let prueba3 = transicion Verde Verde let prueba4 = transicion EnAmarillo Rojo</pre>	Output <pre>type color = EnRojo Verde Rojo EnVerde Amarillo EnAmarillo val transicion : color -> color -> color * string = <fun> val prueba1 : color * string = (EnVerde, "cambiar a amarillo") val prueba2 : color * string = (EnRojo, "cambiar a verde") val prueba3 : color * string = (Verde, "Accion por defecto") val prueba4 : color * string = (EnAmarillo, "cambiar a rojo")</pre>
---	--

Nos devuelve

Output <pre>type color = EnRojo Verde Rojo EnVerde Amarillo EnAmarillo val transicion : color -> color -> color * string = <fun> val prueba1 : color * string = (EnVerde, "cambiar a amarillo") val prueba2 : color * string = (EnRojo, "cambiar a verde") val prueba3 : color * string = (Verde, "Accion por defecto") val prueba4 : color * string = (EnAmarillo, "cambiar a rojo")</pre>
--

OCaml dedujo automáticamente que la función transición recibe dos parámetros de tipo color y devuelve color * string

2) OCaml es un lenguaje orientado a expresiones, lo que quiere decir que casi todo el código se construye mediante expresiones que siempre devuelven un valor.

Este lenguaje tiene estructuras de control como if y match ... with

```
let transicion coloractual cambiara =  
  match (coloractual, cambiara) with  
  | (EnRojo, Verde) -> (coloractual, "cambiar a verde")  
  | (EnVerde, Amarillo) -> (coloractual, "cambiar a amarillo")  
  | (EnAmarillo, Rojo) -> (coloractual, "cambiar a rojo")  
  | _ -> (coloractual, "Accion por defecto")
```

Cada caso con el match devuelve dos valores del tipo color * string como indica el output, ese resultado es el valor retornado por la funcion.

Comparado con lisp, ahí se utilizan estructuras como if o cond para evaluar distintas condiciones, utilizando expresiones booleanas para ver que rama ejecutar.

En OCaml, en cambio, se pueden comparar directamente los “constructores” de un tipo de dato y actuar según el patron encontrado.

```
let timer tiempo_unix =  
  if(tiempo_unix mod 216) <= 90 then "rojo"  
  else if (tiempo_unix mod 216) <= 210 then "verde"  
  else "amarillo"
```

También puede observarse en la función timer que la estructura if devuelve directamente un valor. En OCaml, el if es una expresión y debe devolver un valor en todas sus ramas, con la condición de que el if y el else deben devolver el mismo tipo de dato, si el if devuelve un int y el else un string el programa no compilaría.

Bibliografía:

<https://ocaml.org/industrial-users/businesses>

<https://ocaml.org/manual/5.4/coreexamples.html>